# Capítulo 10. A producción

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Todo lo que pueda salir mal, saldrá mal.*
>
> —Ley de Murphy

Cuando empiezas tu carrera como programador, la producción puede resultar intimidante y emocionante a la vez. La producción es el momento en el que tu código pasa del desarrollo a un entorno real utilizado por personas reales. Es la culminación de toda la planificación, la resolución de problemas y las noches en vela.

La producción representa una de las etapas finales del SDLC. Es donde tu aplicación se enfrentará a su verdadera prueba: dar servicio a usuarios finales reales, manejar datos reales y funcionar bajo cargas de trabajo extensas.

Dejas atrás la red de seguridad de tu entorno de desarrollo local, donde solo estás tú y tu código, y entras en un espacio donde personas

reales dependen de tu trabajo. Cuando surgen problemas, son visibles para todos. Puede que pienses que tu código base es solo otra aplicación más, pero en manos de los usuarios se convierte en una herramienta que impulsa el valor empresarial y genera un cambio significativo en el mundo.

En este capítulo, obtendrás una mejor comprensión y apreciación de la producción. Comenzarás a pensar en crear código listo para la producción en una etapa más temprana del SDLC. Un componente clave de esto es comprender las diferencias fundamentales entre los entornos de desarrollo y producción, y cómo salvar esa brecha de manera eficaz.

## Las complejidades de los entornos de producción

«Houston, tenemos un problema». Estas famosas palabras de la misión Apolo 13, que estuvo a punto de acabar en desastre, nos recuerdan que las cosas pueden salir mal incluso en entornos controlados. Aunque el desarrollo de software rara vez implica cuestiones de vida o muerte, pueden surgir problemas inesperados al pasar a entornos diferentes, especialmente a la producción.

Imagina que tú y tu equipo lleváis meses trabajando en un nuevo producto. Tenéis suficiente cobertura de pruebas y vuestros equipos de control de calidad lo han sometido a duras pruebas en diversos entor-

nos. Obtenéis la aprobación de todos los equipos involucrados y estáis listos para empezar.

Se pulsan todos los botones correctos y tu proyecto emprende su viaje inaugural hacia la producción. Al principio, todo parece ir bien y el tráfico comienza a fluir. Pero, de repente, se desata el caos. Los usuarios informan de errores, aparecen problemas de rendimiento inesperados y tu sistema de monitoreo, junto con tu bandeja de entrada, se inundan de alertas.

¿Qué ha pasado? El código funcionaba en tu máquina. Funcionaba en todos los entornos en los que lo probaste. Si has vivido esta situación, no estás solo. Esto subraya la importancia de la producción. No es solo la parada final de nuestro código. Es el mundo real, la prueba definitiva, donde los usuarios reales interactúan con nuestras aplicaciones.

Aunque tu software pueda funcionar en entornos de prueba, su verdadero valor se determina por su rendimiento en producción con usuarios reales. Las diferencias entre el desarrollo y la producción a menudo se manifiestan de dos maneras: los usuarios interactúan con tu aplicación de formas impredecibles y se produce el famoso «funciona en mi máquina», donde el código se comporta de manera diferente en distintos entornos. Exploremos ambos retos y descubramos algunas estrategias prácticas para salvar esta brecha.

## Los usuarios son impredecibles

Los desarrolladores suelen caer en la trampa de pensar que pueden anticipar cómo interactuarán los usuarios con sus aplicaciones. A ve-

ces es posible que no tengas una visión completa de tu base de usuarios. Otras veces crees saber más de lo que realmente sabes. Ambas situaciones pueden dar lugar a expectativas desalineadas entre desarrolladores y usuarios.

Aunque esto es absolutamente importante para el éxito de un proyecto, no hay ninguna cantidad de pruebas que puedas escribir ni escenarios que puedas simular que puedan replicar la naturaleza impredecible del uso en el mundo real. Por eso los usuarios son tu prueba definitiva.

---

**UNA HISTORIA SOBRE «LOS USUARIOS SON IMPREDECIBLES»**

Aquí Dan. Uno de mis primeros trabajos al salir de la universidad fue como especialista de soporte de nivel 2. Un usuario me llamó para preguntarme por qué el «portavasos» de su ordenador no funcionaba correctamente. Le respondí: «Señor, los ordenadores no tienen portavasos». Resultó que se refería a la unidad de CD-ROM. Este incidente ilustra perfectamente cómo ninguna cantidad de pruebas podría habernos preparado para que un usuario confundiera una unidad de CD-ROM con un portavasos.

---

Los siguientes son factores que contribuyen a la imprevisibilidad de los usuarios:

*Datos del mundo real*

Ni siquiera los datos de prueba diseñados con mayor meticulosidad pueden anticipar las formas creativas en que los usuarios interactuarán con tus formularios. Introducirán datos de formas que solo parecerían plausibles en un guion de Hollywood.

*Escala y concurrencia*

El tráfico del mundo real puede poner de manifiesto problemas de escalabilidad que no eran evidentes en entornos controlados con datos de prueba.

*Entornos diversos*

Los usuarios reales tienen todo tipo de entornos. Esto incluye diferentes sistemas operativos, recursos del sistema, configuraciones de seguridad, proveedores y versiones de navegadores, latencia de red y mucho más.

*Desviación ambiental*

Las diferentes versiones de software o bibliotecas en diferentes entornos pueden provocar un comportamiento inconsistente que no se presentaba durante las pruebas.

*Casos de uso inesperados*

Los usuarios pueden interactuar con tu software de formas que los desarrolladores no habían previsto, lo que revela casos de perímetro o situaciones imprevistas.

Para prepararte para lo desconocido del mundo real, puedes adoptar prácticas como los lanzamientos canario, las pruebas A/B y la observabilidad robusta (monitoreo exhaustivo mediante registros, métricas y

rastreo distribuido). Estos métodos aprovechan el poder de las pruebas de usuarios del mundo real y minimizan el riesgo.

## «Pero funciona en mi máquina»

Recibes una notificación sobre un nuevo problema que se ha registrado en producción. Lees la descripción y te das cuenta de que se trata de una función en la que has trabajado. Acudes a tu portátil para revisar el escenario y ver si puedes reproducir el problema y, por supuesto, no puedes. Funciona en mi máquina, debe ser un error del usuario. «No se puede reproducir» es la frase que utilizan los desarrolladores para cerrar los problemas en estas situaciones.

En el desarrollo de software, «funciona en mi máquina» es una frase que utilizan los desarrolladores cuando su código se ejecuta correctamente en su propio ordenador, pero falla cuando se implementa en otros entornos. Aunque «funciona en mi máquina» puede ayudarte a cerrar rápidamente un informe de errores, no resuelve el problema para el usuario final, y es necesario determinar qué está causando las discrepancias entre el desarrollo y la producción.

Las siguientes son variables que pueden cambiar entre entornos:

*Diferencias de entorno*

> Imagina que has estado desarrollando en tu MacBook Pro con 16 GB de RAM, pero la producción se ejecuta en servidores Linux con diferentes restricciones de memoria. Tu base de datos MySQL local puede ser la versión 8.0, mientras que la

producción utiliza PostgreSQL 14. Estas diferencias en los sistemas operativos, los recursos disponibles, las configuraciones de red, los proveedores de bases de datos y las versiones de dependencias pueden hacer que el código que funciona perfectamente en tu máquina falle estrepitosamente en producción.

*Gestión de la configuración*

En desarrollo, tu aplicación podría conectarse a una base de datos local que se ejecuta en tu ordenador portátil. Es posible que estés registrando todo para ayudarte a depurar problemas, enviando solicitudes a la API de prueba de un proveedor de pagos que utiliza números de tarjetas de crédito falsos. Pero cuando implementas ese mismo código en producción, es necesario que cambie automáticamente a una base de datos segura en la nube, reduzca el registro para capturar solo los errores y se conecte a un procesador de pagos real que cobra dinero real.

*Concurrencia y carga*

Imagina este escenario: has estado probando la función de tu carrito de la compra localmente haciendo clic varias veces en «Añadir al carrito» para asegurarte de que funciona. Pero el Black Friday, miles de clientes pulsan ese mismo botón simultáneamente. De repente, tu base de datos, que gestionaba tus pruebas individuales con facilidad, se ve desbordada por las solicitudes. La memoria de tu servidor, que funcionaba al 30 % de su capacidad, se dispara al 95 % al intentar procesar cientos de sesiones de usuario a la vez. Esta es la diferencia real entre el desarrollo y la producción.

*Diversidad de datos*

Tu base de datos de pruebas local probablemente tenga un puñado de registros limpios y bien formateados que tú mismo has creado. Quizás «John Smith» tenga un número de teléfono perfecto como «555-123-4567» y una dirección de correo electrónico bien formateada. Pero la producción cuenta una historia diferente. Los datos reales de los clientes son desordenados e impredecibles. Es posible que los sistemas heredados hayan almacenado los datos de forma diferente, lo que ahora rompe la lógica de validación. Luego está el problema del volumen. Tus consultas de prueba se ejecutan instantáneamente en cientos de registros, pero la producción tiene millones de clientes que abarcan años de actividad. Por eso es importante realizar pruebas con conjuntos de datos realistas y diversos. Los datos de producción siempre te sorprenderán de alguna manera.

*Dependencias externas*

Las aplicaciones modernas son como los deportes de equipo: ningún jugador va a ganar el partido por sí solo. Tu aplicación puede gestionar la interfaz de usuario, pero pasa a un sistema de procesamiento de pagos para gestionar los pagos. Puede enviar solicitudes a un sistema externo para el envío de correos electrónicos. Puede que dependa de Google para la autenticación de usuarios. Pero en producción, cuando uno de tus «compañeros de equipo» tiene un mal día (el procesador de pagos es lento, el servicio de correo electrónico alcanza los límites de velocidad o el proveedor de autenticación deja de funcionar), tu equipo (aplicación) bien organizado puede seguir perdiendo el partido.

*Restricciones de seguridad*

Los entornos de producción están bloqueados de formas que tu máquina local no lo está. Tu ordenador portátil puede aceptar sin problemas solicitudes HTTP de cualquier lugar, pero los servidores de producción se encuentran detrás de cortafuegos que bloquean el tráfico inesperado. Tus llamadas a la API local funcionan bien con HTTP simple, pero la producción requiere HTTPS con certificados TLS válidos. Esa interfaz que estás probando localmente puede llamar libremente a tu backend, pero la producción aplica políticas CORS que podrían bloquear las mismas solicitudes. Estas capas de seguridad protegen los datos y el dinero de los usuarios reales, pero pueden hacer que tu código falle en producción, incluso cuando funciona perfectamente en tu máquina de desarrollo.

Para salvar la brecha entre «funciona en mi máquina» y «funciona en producción», ten en cuenta lo siguiente:

*Contenedorización*

Utiliza herramientas como Docker para crear entornos coherentes entre el desarrollo y la producción, eliminando la situación de «pero funciona en mi máquina».

*Observabilidad*

Implementa un registro exhaustivo, métricas y rastreo distribuido para comprender cómo se comporta tu aplicación en producción y detectar problemas antes de que afecten a los usuarios.

*Paridad del entorno*

Mantén entornos de prueba que reflejen fielmente la producción, lo que te permitirá detectar tempranamente los problemas específicos del entorno.

*Integración continua e implementación continua*

Practica CI/CD para garantizar que tu código se pruebe regularmente en condiciones similares a las de producción. Más adelante en este capítulo aprenderás más sobre CI/CD.

Recuerda que a tus usuarios no les importa que funcione en tu máquina. Les importa que funcione en la suya. Estas son las complejidades de la producción y algunos consejos sobre cómo salvar la brecha entre los entornos de prueba y la imprevisibilidad de los usuarios reales. En la siguiente sección, aprenderás a evitar algunos de estos problemas teniéndolos en cuenta a lo largo del ciclo de vida del desarrollo.

## Creación de código listo para la producción

Si eres padre y te estás preparando para enviar a tu hijo a su primer día de colegio, tienes que pensar en muchos detalles. No te limitarías a despertarlo, darle su mochila y empujarlo hacia la puerta, ¿verdad? Por supuesto que no. Te asegurarías de que coman bien, vistan adecuadamente, tengan todo lo necesario y sepan cómo ponerse en contacto contigo si algo sale mal. Los prepararías para el mundo al que se enfrentan.

Lo mismo se aplica a la hora de preparar tu código para la producción. Tu código es como tu hijo, y debes asegurarte de que esté listo para afrontar los retos imprevistos del mundo real. Al igual que los padres no pueden predecir todas las situaciones a las que se enfrentará su hijo, los desarrolladores no pueden anticipar todos los problemas que pueden surgir en producción. Sin embargo, con una preparación adecuada, puedes dar a tu código las mejores posibilidades de éxito teniendo en cuenta la optimización del rendimiento, las configuraciones específicas del entorno, la gestión y el registro de errores, y los aspectos esenciales de seguridad a lo largo del proceso de desarrollo.

## Optimización del rendimiento

La optimización del rendimiento no es un dial que simplemente se gira cuando se quiere que las cosas funcionen sin problemas. Es una mentalidad que debes adoptar antes de escribir cualquier código, y debe ser un área de tu aplicación que revises constantemente a lo largo del proceso de desarrollo.

Si bien el rendimiento puede consistir en hacer que tu código se ejecute más rápido, en última instancia se trata de proporcionar la mejor experiencia posible a tus usuarios. Puedes emplear varias estrategias clave para mejorar el rendimiento. Piensa en ellas como bloques de construcción: no es necesario que las implementes todas a la vez, sino que puedes incorporarlas gradualmente a tu proceso de desarrollo:

*Programación asíncrona*

En lugar de esperar a que una tarea se complete antes de pasar a la siguiente (ejecución sincrónica), la programación asincrónica permite que las tareas se ejecuten en segundo plano y notifican al programa principal cuando se completan. Esta es una solución habitual para este problema, pero hay que utilizarla con precaución, ya que introduce sus propias complejidades y compensaciones.

*Reducir las llamadas de red*

Cada solicitud que realiza tu aplicación tiene un coste asociado, y la moneda de cambio es el tiempo y los recursos. En lugar de realizar múltiples llamadas más pequeñas, considera la posibilidad de agruparlas en una sola solicitud.

*Estrategias de almacenamiento en caché*

Si puedes identificar los datos a los que se accede con frecuencia y que no cambian, puedes almacenarlos en la memoria o en algún tipo de sistema de almacenamiento rápido. Esto reducirá la carga de la base de datos y aumentará los tiempos de respuesta. Puedes obtener más información sobre las ventajas e inconvenientes del almacenamiento en caché en el capítulo 8.

*Optimización de consultas a bases de datos*

Utiliza consultas eficientes, una indexación adecuada y evita los problemas de consultas N + 1.

*Minimización de código*

En el desarrollo frontend moderno, la minificación de código elimina los caracteres innecesarios de tu código sin cambiar su funcionalidad. Piensa en ello como eliminar todos los espacios y saltos de línea de un libro manteniendo todas las palabras: el significado sigue siendo el mismo, pero ocupa menos espacio. La

minificación elimina caracteres como espacios en blanco, saltos de línea y comentarios para que los archivos sean más pequeños y se descarguen más rápido.

*Agrupación de código*

La agrupación combina varios archivos de código separados en unos pocos archivos más grandes. En lugar de descargar 20 archivos JavaScript separados, tu navegador descarga 1 o 2 archivos agrupados. Las herramientas de compilación modernas, como Webpack o Vite, pueden realizar la agrupación junto con otras optimizaciones, como la eliminación de código no utilizado (tree shaking) y la minificación.

*Carga diferida*

Carga los recursos (imágenes, fragmentos de código, componentes) solo cuando sean realmente necesarios, en lugar de hacerlo por adelantado. Esto reduce el tiempo de carga inicial de la página.

*Redes de distribución de contenidos (CDN)*

Las CDN complementan las optimizaciones, como la minificación y la agrupación, al servir tus activos desde ubicaciones físicamente más cercanas a tus usuarios, lo que puede reducir significativamente los tiempos de carga.

La optimización del rendimiento no consiste solo en hacer que el código se ejecute más rápido, sino en proporcionar la mejor experiencia posible a los usuarios. Para ello, puedes cambiar tu mentalidad y pensar en algunas de las optimizaciones que has aprendido en esta sección.

## Configuraciones específicas del entorno

Es posible que hayas oído la frase «la configuración es código», pero, a diferencia del código, las configuraciones varían según el entorno. La configuración es similar a ajustar los parámetros de la cámara de tu teléfono para diferentes condiciones de iluminación. Del mismo modo que los ajustes optimizados para la luz solar intensa pueden fallar en condiciones de poca luz, tu aplicación necesita configuraciones personalizadas para funcionar de forma fiable en cada entorno específico.

Veamos un antipatrón común y su solución. En el siguiente ejemplo, el desarrollador ha codificado de forma rígida las credenciales de la base de datos para su máquina local. En lugar de codificar de forma rígida las credenciales de esta manera, es mejor optar por una configuración que se pueda establecer para cada entorno:

```
// DON'T DO THIS
public class DbConnection {
   private final String url = "jdbc:postgresql://localhost:5432/fose";
   private final String username = "admin";
   private final String password = "secret";
}

// DO THIS INSTEAD
public class DbConnection {
   private final String url;
   private final String username;
   private final String password;

   public DbConnection(AppConfiguration config) {
```

```
        this.url = config.getDatabaseUrl();
        this.username = config.getDatabaseUsername();
        this.password = config.getDatabasePassword();
    }
}
```

Acabas de ver un ejemplo de codificación fija de una cadena de conexión a la base de datos específica para un entorno de desarrollo. Esto no se adapta a medida que trasladas tu código a diferentes entornos. A continuación se muestran algunos ejemplos más de configuraciones que pueden cambiar según el entorno:

- Configuración de registro
- Puntos finales de servicios externos
- Configuración de la base de datos
- Configuración de caché
- Configuración de correo electrónico
- Configuración de seguridad
- Indicadores de funciones y comportamiento de la aplicación

Ahora que sabes que esto es algo que debes tener en cuenta, veamos algunas formas de configurar estos elementos en diferentes entornos.

## Archivos de configuración

Podemos utilizar archivos de configuración para configurar todo, desde cadenas de conexión de bases de datos hasta claves API y ajustes específicos de la aplicación. Aunque se trata de código, suelen ser legibles para los humanos, lo que los hace muy atractivos. Los archivos de

configuración también te ofrecen la opción de tener un archivo para cada entorno en el que realices la implementación.

En el siguiente ejemplo, definimos dos archivos de configuración: uno para desarrollo y otro para producción. Esto te permite definir las propiedades de cada entorno:

```
# application.yaml (dev)
services:
 service1:
   url: http://localhost:8080
   api-key: 123456
database:
 url: jdbc:postgresql://localhost:5432/fose
 username: postgres
 password: postgres
logging:
 level: INFO

# application-prod.yaml
services:
 service1:
   url: http://real-service1.com
   api-key: 789012
database:
 url: jdbc:postgresql://fosedb.com:5432/fose
 username: postgres
 password: f@3ser#4il_m$%@$LW0
logging:
 level: ERROR
```

### Variables de entorno

Si prestaste atención a los archivos de configuración de la sección anterior, es posible que hayas notado que contienen información confidencial. Aunque estos archivos de configuración son específicos, siguen planteando problemas. El primer problema es que estás exponiendo información confidencial que se registrará en el control de versiones. El otro problema es que la información confidencial, como las claves API, suele rotarse a intervalos programados. Esto significa que, para cambiar una clave API, tendrás que volver a implementar toda la aplicación, lo que no es lo ideal. En este ejemplo, la información confidencial ahora se define en una variable de entorno y ya no está codificada de forma fija:

```
# application-prod.yaml
services:
 service1:
   url: ${SERVICE1_URL}
   api-key: ${SERVICE1_API_KEY}
```

### Indicadores de características

*Los indicadores de características* (, también conocidos como *conmutadores de características*) son una técnica de desarrollo de software que permite a los desarrolladores habilitar o deshabilitar características en tiempo de ejecución sin implementar código nuevo. Considérelos como un panel de control para las características de tu aplicación.

Los indicadores de características ofrecen varias ventajas tanto para tus equipos de desarrollo como para tus usuarios. Puedes enviar código a producción mientras mantienes las características ocultas a los usuarios, lo que permite implementaciones más seguras y una mejor separación entre las versiones de código y las versiones de características. Permiten implementaciones graduales en grupos de usuarios específicos, lo que te permite probar las características con un subconjunto de usuarios antes de la implementación completa. Los indicadores de características también facilitan las pruebas A/B, ya que permiten comparar diferentes experiencias de usuario, y ofrecen la posibilidad de revertir rápidamente las características sin necesidad de implementar código.

Hay tres tipos comunes de indicadores de características que debes tener en cuenta. *Los indicadores de lanzamiento* te ayudan a ocultar las características incompletas en producción, manteniendo tu rama principal implementable mientras continúa el trabajo. *Los indicadores de experimentación* están diseñados para comparar las experiencias de los usuarios y recopilar datos sobre su comportamiento. *Los indicadores de permiso* controlan el acceso a las características en función de los roles de los usuarios o los niveles de suscripción, lo que los hace útiles para ofertas de productos por niveles.

CONSEJO

Las banderas de características deben revisarse y limpiarse periódicamente. Las banderas temporales que se convierten en permanentes crean una complejidad innecesaria en tu código base. Establece un calendario de limpieza que funcione bien para tu equipo y fija fechas de caducidad al crear nuevas banderas de características. Crea documentación que indique claramente el propósito y la vida útil prevista de cada bandera.

---

## Secretos

*Los secretos* son valores de configuración sensibles e es, como claves API, contraseñas y tokens, que requieren un cuidado especial. Gestionar los secretos correctamente es fundamental para la seguridad de las aplicaciones y requiere seguir varias prácticas importantes.

Nunca almacenes secretos directamente en tu código ni los envíes al control de versiones. Esta violación de la seguridad puede costarte dinero y dañar la confianza de los usuarios. Si los envías al control de versiones, tendrás que reescribir el historial de Git, lo que puede ser un proceso complejo. Además, compartir el código fuente con terceros se convierte en un riesgo de seguridad significativo cuando los secretos están incrustados en la base de código.

En su lugar, utiliza servicios de gestión de secretos dedicados como AWS Secrets Manager, HashiCorp Vault o Azure Key Vault. Estos servicios proporcionan almacenamiento seguro, cifrado y controles de acceso diseñados específicamente para datos confidenciales. Haz que la rotación de credenciales sea una práctica habitual, idealmente automati-

zada a través de tu plataforma de gestión de secretos. Esto limita la ventana de vulnerabilidad si las credenciales se ven comprometidas.

El acceso a los secretos de producción debe controlarse estrictamente y limitarse a los miembros del equipo que lo necesiten absolutamente. Utiliza el principio del mínimo privilegio e implementa controles de autenticación y autorización adecuados. Por último, mantén secretos separados para cada entorno en el que se publique la aplicación. El desarrollo, la puesta en escena y la producción nunca deben compartir las mismas credenciales. Este aislamiento evita la exposición accidental de los entornos de producción durante el desarrollo y las pruebas.

Como puedes ver, hay una serie de aspectos que debes tener en cuenta al configurar tu aplicación para múltiples entornos. La siguiente consideración para crear código listo para producción es emplear un manejo de errores y un registro adecuados para detectar los problemas cuando surjan.

## Gestión de errores y registro

Como ya has aprendido, el mundo de la producción puede ser un lugar impredecible. La ley de Murphy nos recuerda que, si algo puede salir mal, saldrá mal. En este entorno implacable, el manejo y registro adecuados de errores no son solo buenas prácticas, sino habilidades esenciales para la supervivencia de las aplicaciones de producción. Nos permiten detectar problemas antes de que se conviertan en fallos en todo el sistema y proporcionan información crítica a la hora de resolver los problemas que inevitablemente surgirán.

Cuando empiezas a crear una nueva función, es natural centrarse en la «ruta feliz». Se trata del escenario ideal en el que todo funciona a la perfección para tus usuarios. Sin embargo, en el mundo real del desarrollo de software, esto solo representa una pequeña parte de lo que debes tener en cuenta. Siguiendo el principio de Pareto, aunque la ruta feliz puede representar el 20 % de tu código, el manejo de casos extremos y errores a menudo representa el 80 % de tu tiempo y esfuerzo. Aquí es donde pasarás la mayor parte de tu tiempo de desarrollo y donde los desarrolladores experimentados se distinguen.

## Gestión de errores

Entonces, ¿cómo es en la práctica un manejo de errores eficaz ? A continuación, se presentan algunas estrategias concretas que utilizarás con regularidad:

*Degradación elegante*

Cuando una función falle, proporciona una versión simplificada en lugar de interrumpirla por completo. Por ejemplo, si tu API meteorológica no funciona, muestra los datos meteorológicos almacenados en caché con una nota que indique que pueden estar desactualizados, en lugar de mostrar una página de error.

*Mensajes de error significativos*

Reemplaza la jerga técnica por un lenguaje fácil de entender para los usuarios. En lugar de «Error en la conexión con la base de datos», diles a los usuarios «Estamos teniendo problemas para cargar tus datos en este momento. Vuelve a intentarlo en unos minutos».

*Registro para la depuración*

Registra lo que ha fallado entre bastidores para poder solucionarlo más tarde. Registra los detalles técnicos (como códigos de error y trazas de pila) que ayudan a los desarrolladores, mientras que a los usuarios solo les muestras lo que necesitan saber.

*Opciones alternativas*

Ten siempre un plan B. Si tu procesador de pagos principal falla durante el proceso de pago, cambia automáticamente a un procesador de respaldo para que los clientes puedan completar sus compras.

## Registro

Un registro eficaz proporciona información valiosa sobre los problemas de producción. El reto consiste en encontrar el equilibrio adecuado. Un registro insuficiente puede dejarte en la oscuridad, mientras que un registro excesivo puede crear ruido que oculte información crucial.

Aunque las implementaciones pueden variar según los lenguajes de programación, el concepto de niveles de registro es común. A continuación se muestran los niveles de registro en Java que ayudan a los desarrolladores a categorizar y filtrar los mensajes de registro en función de su gravedad e importancia:

*ERROR*

Úsalo para problemas graves que requieran atención inmediata.

*ADVERTENCIA*

: Para situaciones potencialmente peligrosas.

*INFO*

: Eventos normales del proceso empresarial

*DEBUG*

: Información detallada para la depuración

Al registrar, intenta no ser demasiado prolijo. Céntrate en los elementos críticos. No debes registrar lo siguiente:

- Información confidencial, como contraseñas, claves API, tokens de autenticación o secretos.
- Información de identificación personal (PII). Obtendrás más información al respecto más adelante en este capítulo.
- Objetos grandes o cargas útiles de respuesta completas. En su lugar, registra información significativa que ayude a identificar el problema en cuestión.
- Registro duplicado de la misma información en varias capas de tu aplicación.

Como acabas de aprender, el manejo de errores y el registro son vitales para obtener visibilidad cuando las cosas salen mal en la producción. Otro aspecto importante de la creación de código listo para la producción es asegurarse de que sea seguro.

# Fundamentos de seguridad

La seguridad no es algo en lo que piensas una vez y sigues adelante con tu vida. Es un proceso continuo que requiere vigilancia y atención regular. A medida que implementas aplicaciones en producción, recuerda que la seguridad debe integrarse en tu proceso de desarrollo desde el principio, no añadirse al final. Aunque no es necesario que seas un experto en seguridad, comprender estos fundamentos te ayudará a crear aplicaciones más seguras y te convertirá en un miembro más valioso del equipo. Un recurso realmente bueno para aprender sobre la seguridad de las aplicaciones web es el Proyecto Abierto de Seguridad de Aplicaciones Web ( OWASP). OWASP ofrece orientación gratuita y práctica sobre seguridad que goza de la confianza de desarrolladores de todo el mundo. En esta sección, aprenderás algunas formas de hacer que tus aplicaciones sean más seguras.

## Proteger la comunicación con HTTPS

Mientras desarrollas un e localmente, es posible que te inclines por utilizar el protocolo de transferencia de hipertexto (HTTP), ya que, francamente, es más cómodo. Sin embargo, es importante realizar pruebas con HTTP Secure (HTTPS) antes de implementar en producción. Existen herramientas que te permiten crear certificaciones de confianza para el desarrollo local.

Hay ciertos mensajes que pueden ver todos, y hay mensajes que solo quieres que vea el destinatario. Imagina una postal que envías a alguien mientras estás de vacaciones. La postal tiene una foto de la playa en el anverso y en el reverso un mensaje escrito a mano que cualquie-

ra puede ver. Esto es lo que ocurre cuando se envían datos a través de HTTP. HTTP transfiere datos de texto sin formato entre tu navegador y el servidor. Cualquier información confidencial, como contraseñas, números de tarjetas de crédito, secretos o datos personales, es visible para cualquiera que intercepte el tráfico.

Si quieres mantener la privacidad de tu mensaje de vacaciones, lo pondrías dentro de un sobre sellado; esto es lo que hace HTTPS por ti. HTTPS añade una capa de cifrado utilizando protocolos de seguridad de la capa de transporte (TLS) .

Los certificados TLS son como tarjetas de identificación digitales para sitios web. Cuando visitas un sitio HTTPS, el servidor presenta su certificado para demostrar su identidad. Estos certificados son emitidos por autoridades de certificación (CA) de confianza que verifican la identidad del propietario del sitio web.

Una vez que tu navegador verifica el certificado, TLS utiliza esta conexión de confianza para establecer un cifrado sólido para toda la transferencia de datos. Este cifrado garantiza que tu información no pueda ser leída por miradas indiscretas y también significa que la información no puede ser modificada mientras se transmite entre tu navegador y el servidor.

NOTA

Aunque el protocolo capa de sockets seguros (SSL) es un protocolo antiguo que ha sido sustituido por TLS, mucha gente sigue utilizando el término «SSL» por costumbre. SSL tiene vulnerabilidades de seguridad conocidas y no debe utilizarse en aplicaciones de producción. Cuando alguien se refiere a SSL hoy en día, normalmente se refiere a TLS, pero es importante verificar que los sistemas realmente utilizan protocolos TLS.

---

HTTPS no va a solucionar todos tus problemas de seguridad. Es una capa de seguridad que debes tener en cuenta al poner en producción tus aplicaciones. A continuación te indicamos algunas buenas prácticas que puedes seguir:

- Utiliza HTTPS en todas partes, no solo en las páginas de inicio de sesión.
- Redirige HTTP a HTTPS automáticamente.
- Mantén los certificados actualizados (considera la posibilidad de utilizar herramientas de gestión de certificados automatizadas).
- Utiliza versiones TLS seguras (1.2 o superior).

## Buenas prácticas de autenticación

Las brechas de seguridad pueden causar muchos problemas dentro de tu organización, como la exposición de datos confidenciales, el daño a la confianza de los clientes y, potencialmente, pérdidas financieras y de reputación significativas. Una autenticación adecuada es una buena defensa contra estas amenazas.

*La autenticación* es el proceso de verificar que los usuarios son quienes dicen ser, mientras que *la autorización* determina lo que los usuarios verificados pueden hacer. La primera y más importante regla es: nunca, bajo ninguna circunstancia, crees tus propias medidas de seguridad. Aunque crear tus propios sistemas de autenticación puede parecer sencillo, los expertos en seguridad han dedicado décadas a identificar y mitigar vulnerabilidades complejas que no siempre son evidentes.

Spring Security en el mundo Java representa más de 20 años de trabajo en materia de seguridad. Ofrece soluciones probadas para problemas de seguridad comunes, como el almacenamiento de contraseñas. Incluye codificadores de contraseñas que evitan el almacenamiento de contraseñas como texto sin formato, lo cual es muy peligroso. En el siguiente ejemplo, creamos un nuevo `BCryptPasswordEncoder`, que se utilizará para codificar contraseñas:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

A continuación, utilizamos ese codificador de contraseñas para codificar la contraseña antes de guardarla en una base de datos:

```
// Create new user entity
User user = new User();
user.setUsername(registrationDto.getUsername());
```

```
    user.setEmail(registrationDto.getEmail());

    // Encode the password before saving
    user.setPassword(passwordEncoder.encode(registrationDto.getPassword()));
    // Save and return the new user
    return userRepository.save(user);
```

La autenticación es un componente vital para la seguridad de las aplicaciones, ya que constituye la primera línea de defensa contra el acceso no autorizado. Exploremos los componentes básicos de un sistema de autenticación robusto.

*Una gestión sólida de las contraseñas* es esencial para proteger tus sistemas contra el acceso no autorizado. Por ejemplo, exige contraseñas con un mínimo de 12 caracteres, que combinen mayúsculas, minúsculas, números y caracteres especiales. Comprueba las bases de datos de contraseñas comunes o comprometidas y evita que los usuarios reutilicen contraseñas anteriores. Utiliza algoritmos de hash modernos con sales únicas para cada contraseña. Una *sal* es un dato aleatorio que se añade antes del hash para evitar los ataques de tabla arcoíris. Fomenta y apoya el uso de gestores de contraseñas con capacidad de autocompletar.

Incluso con todas estas protecciones técnicas, recuerda que las contraseñas dependen en última instancia del comportamiento humano. La política de contraseñas más sólida no servirá de nada si los usuarios escriben las contraseñas en notas adhesivas o las comparten con sus compañeros. Por eso es tan importante implementar múltiples capas

de seguridad (como la autenticación multifactorial, que se tratará más adelante).

*La protección de las cuentas de usuario* requiere múltiples capas de defensa contra los intentos de acceso no autorizado. Por ejemplo, implementa un retroceso exponencial para los intentos de inicio de sesión fallidos, comenzando con retrasos cortos, como 5 segundos, y aumentando gradualmente hasta 30 minutos o más. Realiza un seguimiento de los intentos fallidos en varias cuentas desde la misma dirección IP y realiza un monitoreo cuando se esté atacando a cuentas individuales desde múltiples ubicaciones. Añade CAPTCHA después de varios intentos fallidos para evitar ataques automatizados y alerta a los usuarios cuando se produzcan intentos de inicio de sesión desde nuevos dispositivos o ubicaciones desconocidas. Por último, implementa flujos seguros de recuperación de cuentas con pasos de verificación adecuados para ayudar a los usuarios legítimos a recuperar el acceso y mantener alejados a los atacantes.

*Una gestión eficaz de las sesiones* protege a los usuarios mientras interactúan con tu sistema y evita el acceso no autorizado a las sesiones activas. Utiliza tokens criptográficamente seguros con suficiente entropía para garantizar que las sesiones no puedan adivinarse o secuestrarse fácilmente. Implementa tiempos de espera de sesión tanto por inactividad como absolutos. Por ejemplo, cierra automáticamente la sesión de los usuarios después de 30 minutos de inactividad o 8 horas en total, con tiempos de espera más cortos para operaciones sensibles como la banca. Mantén una lista de bloqueo de tokens revocados hasta su vencimiento natural para evitar la reutilización de sesiones comprometidas.

*Añadir múltiples capas de verificación* refuerza la seguridad de tu autenticación más allá de las simples contraseñas. Admite múltiples métodos de autenticación, como aplicaciones de autenticación, claves de seguridad y datos biométricos, para ofrecer a los usuarios opciones que se adapten a su situación. Aunque los SMS y el correo electrónico son cómodos y se utilizan mucho, ten en cuenta que son más vulnerables a ataques como el intercambio de tarjetas SIM.

Puedes activar una verificación adicional basada en comportamientos inusuales. Por ejemplo, exige una autenticación adicional cuando los usuarios inicien sesión desde nuevas ubicaciones o dispositivos. Exige la autenticación multifactorial (MFA) durante los procesos de recuperación de cuentas para evitar que los atacantes eludan tu seguridad. Proporciona códigos de respaldo seguros para que los usuarios no se queden bloqueados cuando su método MFA principal no esté disponible. Considera la posibilidad de permitir el almacenamiento en caché de dispositivos de confianza con los controles de seguridad adecuados para que los usuarios no tengan que pasar constantemente por el proceso de autenticación en sus dispositivos habituales.

## Protección de los datos de los usuarios

Tus usuarios te confían sus datos, y tú tienes la responsabilidad profesional y ética de proteger esa confianza. Más allá de los requisitos legales, respetar la privacidad y la seguridad de los datos de los usuarios es simplemente lo correcto. Tus decisiones como desarrollador pueden tener un impacto real en la vida, la seguridad financiera y la seguridad personal de las personas.

Cuando empieces a crear aplicaciones que manejan datos de usuarios, es muy probable que te encuentres con el término*«información de identificación personal* » ( ,*PII*). Este tipo de datos puede incluir detalles obvios, como el nombre y el número de la seguridad social, pero también otros menos obvios, como direcciones IP o identificadores de dispositivos. Algunos datos se consideran PII cuando se combinan con otros datos.

Una forma de empezar a pensar en esto durante el desarrollo es marcar los datos como confidenciales. En el siguiente ejemplo, escrito en Java, una anotación personalizada marca los datos como PII. Dentro de esa anotación personalizada, puedes utilizar varios métodos de enmascaramiento:

```
public class UserProfile {
   private String userId;
   private String hashedPassword;
   @Sensitive  // Custom annotation to mark PII
   private String email;
   @Sensitive
   private String phoneNumber;
   @Sensitive
   private LocalDate dob;
}
```

## Cifrado: tu última línea de defensa

¿Qué ocurre cuando tienes datos confidenciales, como información financiera (tarjetas de crédito, números de cuentas bancarias, declara-

ciones de impuestos, etc.) y necesitas almacenarlos en una base de datos? Se trata de información que no debes almacenar en texto plano bajo ningún concepto. Durante el desarrollo, debes empezar a pensar en el cifrado y considerarlo tu red de seguridad. Piensa en ello como una caja fuerte segura: aunque alguien la abra, no podrá utilizar lo que encuentre sin la llave.

La siguiente clase proporciona un método seguro para cifrar datos confidenciales en Java. Utiliza el Estándar de cifrado avanzado (AES) de , un algoritmo de cifrado de alta seguridad que proporciona protección tanto de la confidencialidad como de la integridad de los datos. Al implementar AES, deberás elegir una longitud de clave; actualmente, 128 bits proporcionan una seguridad sólida para la mayoría de las aplicaciones, aunque muchos desarrolladores están pasando a claves de 256 bits para estar preparados para el futuro ante el avance de la potencia informática:

```
class DataEncryptionService {
  private final SecretKey key;
  DataEncryptionService(SecretKey key) { this.key = key; }

  String encrypt(String s) throws GeneralSecurityException {
    byte[] iv = new byte[12]; new SecureRandom().nextBytes(iv);
    Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
    c.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
    byte[] ct = c.doFinal(s.getBytes(StandardCharsets.UTF_8));
    byte[] out = ByteBuffer.allocate(iv.length + ct.length)
                 .put(iv).put(ct).array();
    return Base64.getEncoder().encodeToString(out); // encodes IV||ciphertext
```

```
    }
}
```

### Requisitos de cumplimiento

*El cumplimiento normativo del software* implica seguir reglas, estándares y regulaciones específicos. Esto incluye tanto las políticas internas de la empresa como los requisitos legales externos. A medida que escribes código, ten en cuenta estas prácticas de cumplimiento:

- Comienza con la privacidad desde el diseño: incorpora la protección de datos en tus sistemas desde el principio.
- Aplica normas de codificación seguras en tu trabajo diario.
- Implementa un cifrado adecuado para los datos confidenciales, tanto en tránsito como en reposo:
  - En tránsito: utiliza HTTPS para todas las transmisiones de datos (esto protege los datos mientras se transmiten entre el usuario y el servidor).
  - En reposo: cifra los datos confidenciales almacenados en tu base de datos (esto protege los datos incluso si alguien obtiene acceso a la base de datos).
  - Ten mucho cuidado al descifrar datos en la memoria del servidor, ya que los atacantes expertos podrían encontrar formas de acceder a ellos.
- Crea mecanismos claros para las solicitudes de eliminación y exportación de datos.
- Documenta todas las interacciones con información confidencial.

No es necesario que memorices todas las normativas, pero debes reconocer cuándo tu código afecta a datos regulados y pedir ayuda a alguien de tu equipo. La seguridad es un esfuerzo de equipo. Aunque implementes las buenas prácticas de seguridad, los profesionales de seguridad de tu organización deben participar en las decisiones de diseño desde el inicio de cada proyecto. Como desarrollador, no siempre reconocerás cuándo te estás acercando a algo que podría crear riesgos de seguridad, y eso es normal.

A continuación se describen algunas normas clave de privacidad que los desarrolladores deben conocer:

*Reglamento General de Protección de Datos (Unión Europea) y Ley de Privacidad del Consumidor de California (California)*

Principales normativas de privacidad que afectan a la forma en que gestionamos los datos de los usuarios. Estas normativas exigen esencialmente lo siguiente:

- Consentimiento claro del usuario para la recopilación de datos
- Posibilidad de eliminar los datos de los usuarios («derecho al olvido»)
- Minimización de datos (recopila solo lo que necesites)
- Manejo adecuado de los datos y medidas de seguridad

*Ley de Portabilidad y Responsabilidad del Seguro Médico (HIPAA)*

Si tu aplicación maneja cualquier tipo de información sanitaria (historiales médicos, datos de seguros, incluso seguimiento de actividad física), se aplica la HIPAA. Las consideraciones clave para los desarrolladores incluyen lo siguiente:

- Cifrar todos los datos sanitarios tanto en el almacenamiento como en la transmisión.
- Implementar controles de acceso estrictos: los usuarios solo deben ver su propia información sanitaria.
- Mantener registros de auditoría detallados de quién ha accedido a qué datos sanitarios y cuándo.
- Asegúrate de que cualquier servicio de terceros que integres también cumpla con la HIPAA.
- Nunca almacene información sanitaria más tiempo del necesario para los fines de su aplicación.

*Industria de tarjetas de pago (PCI)*

Si tu aplicación maneja datos de tarjetas de crédito, deberás cumplir los requisitos de la Norma de Seguridad de Datos de la Industria de Tarjetas de Pago (PCI DSS). Aunque la norma completa es compleja, estos son los puntos clave que debes tener en cuenta como desarrollador:

- Nunca almacene datos de autenticación confidenciales (como códigos CVV).
- Cifrar los datos de los titulares de tarjetas durante la transmisión y el almacenamiento.

- Implementa controles de acceso estrictos.
- Mantén sistemas y aplicaciones seguros.

*Lista de materiales de software (SBOM)*

> Una SBOM funciona como una lista de ingredientes para tu aplicación. Documenta todas las bibliotecas, marcos y dependencias que utiliza tu código, junto con sus números de versión. Piensa en ella como un inventario de todos los componentes básicos de tu software. A efectos de cumplimiento normativo, las SBOM son importantes porque las vulnerabilidades de seguridad aparecen con frecuencia en las bibliotecas más populares. Con una SBOM, puedes identificar rápidamente si tu aplicación utiliza componentes afectados y actualizarlos de forma prompt. La mayoría de las herramientas de desarrollo generan SBOM automáticamente, lo que elimina la necesidad de mantenimiento manual.

---

**ADVERTENCIA**

Registra siempre las acciones relacionadas con la seguridad, pero nunca los datos confidenciales. En caso de duda, consulta a tu equipo de seguridad para que te oriente. Crea un calendario de mantenimiento para actualizar tus dependencias. Establece una cadencia regular (mensual o trimestral) para revisar tu SBOM en busca de vulnerabilidades conocidas y actualizar a las últimas versiones seguras de tus bibliotecas.

---

Recuerda que la seguridad se basa en capas. Ninguna medida de seguridad es perfecta por sí sola, pero la implementación de múltiples ca-

pas de seguridad dificulta considerablemente que los atacantes comprometan tu sistema. Esto es algo en lo que debes pensar durante el proceso de desarrollo, así que intenta aplicar capas de seguridad para adquirir el buen hábito de ser consciente de la seguridad.

Ahora que ya conoces algunos aspectos esenciales de seguridad que debes tener en cuenta a la hora de crear código listo para la producción mientras aún se encuentra en fase de desarrollo, es el momento de hablar del proceso de implementación en sí.

# Proceso de implementación

*Un proceso de implementación e e* es tu hoja de ruta para trasladar tu código de forma segura y fiable de un entorno a la producción. Piensa en él como una cadena de montaje bien engrasada en la que cada componente desempeña un papel específico en la transformación de tu código en una aplicación en funcionamiento a la que pueden acceder los usuarios reales.

Tu canalización de implementación consta de cuatro componentes interconectados que funcionan juntos como los engranajes de una máquina:

*Entornos de implementación*

Estas son las etapas en las que tu código vivirá y se ejecutará, desde tu configuración de desarrollo local hasta los servidores de producción donde los clientes interactúan con tu aplicación.

Cada entorno tiene un propósito específico en la validación de tu código antes de que llegue a los usuarios.

*Estrategias de control de versiones*

Coordina cómo varios desarrolladores aportan cambios al mismo código base sin pisarse unos a otros. Estas estrategias se convierten en tu red de seguridad para gestionar lanzamientos y manejar emergencias.

*Automatización de la implementación*

Elimina los pasos manuales y propensos a errores de mover código entre entornos. En lugar de seguir una lista de verificación y esperar no pasar nada por alto, la automatización garantiza que se ejecute el mismo proceso fiable cada vez.

*CI/CD (integración continua e implementación continua)*

Une todo moviendo automáticamente tu código a través del proceso, desde el momento en que confirmas los cambios hasta que se ejecutan en producción.

Al final de este capítulo, comprenderás cómo encajan estas piezas para crear un proceso de implementación fiable y sin estrés.

## Entornos de implementación

Antes de que puedas comprender el proceso de implementación dentro de tu empresa, es fundamental que entiendas los entornos en los que vas a implementar el código. Estos variarán entre organizaciones

e incluso entre equipos dentro de la misma organización. A continuación, se incluye una lista de entornos comunes que puedes encontrar y sus propósitos:

*Desarrollo local*

Este es tu espacio de trabajo local, donde comienza tu viaje por el mundo de la programación. Aunque se trata de tu espacio de trabajo personal para experimentar, recuerda que tus compañeros desarrolladores comparten una configuración similar.

*Pruebas/control de calidad*

Aunque es de esperar que dispongas de un conjunto completo de pruebas para ejecutar localmente, el entorno de pruebas es tu primer punto de control compartido. Aquí es donde se lleva a cabo el control de calidad, y es posible que tu conjunto de pruebas se comporte de forma diferente a como lo hace localmente.

*Puesta en escena*

Piensa en la puesta en escena como el ensayo general de un gran espectáculo. Es un entorno similar al de producción que sirve como campo de pruebas final antes de la puesta en marcha. Es ideal para pruebas de rendimiento y pruebas de aceptación de los usuarios.

*Producción*

Esta es la etapa principal y tu objetivo final. Aquí, los usuarios reales interactuarán con la aplicación en cuya creación has invertido tanto tiempo y energía.

Anteriormente en este capítulo, aprendiste sobre la gestión de la configuración. Una parte crucial del traslado de una aplicación entre entornos es garantizar la configuración correcta para cada etapa. La gestión de la configuración debe ser una parte integral de tu proceso de implementación, no una idea de último momento. Para comprender cómo trasladar el código de un entorno al siguiente, debes comprender qué cambiará entre esos entornos.

## Estrategias de control de versiones

Las estrategias de control de versiones cobran especial importancia en entornos de producción, donde la estabilidad del código, la fiabilidad y la eficiencia de la implementación son fundamentales. En producción, estas estrategias permiten a los equipos gestionar el lanzamiento de funciones sin interrumpir los servicios en vivo, abordar rápidamente las implementaciones problemáticas (ya sea volviendo a una versión anterior o avanzando con una corrección) y mantener registros de auditoría detallados para fines de cumplimiento.

Probablemente tengas cierta experiencia en el control de versiones gracias a tu formación o tu trabajo. Es posible que lo hayas utilizado en proyectos personales y que conozcas bien los conceptos básicos de Git, como clonar repositorios, añadir y eliminar archivos y enviar cambios. Esta base es ideal para empezar, pero a medida que te adentras en un entorno organizativo, la introducción de equipos aporta nuevas complejidades al control de versiones.

A continuación se presentan algunos flujos de trabajo populares para manejar estos problemas:

*Git Flow*

Utiliza múltiples ramas de larga duración ( ) para separar diferentes tipos de trabajo (funcionalidades, lanzamientos, correcciones). Esto proporciona un fuerte aislamiento entre el código de desarrollo y el de producción. En esta sección analizaremos Git Flow más detenidamente.

*GitHub Flow*

Un enfoque más sencillo que utiliza solo una rama principal y ramas de características de corta duración. Los cambios pasan directamente de las ramas de características a la producción después de su revisión.

*Desarrollo basado en troncos*

Hace hincapié en la integración frecuente, en la que todos los desarrolladores trabajan en una única rama principal con ramas de características de muy corta duración (a menudo menos de un día).

*Tren de lanzamientos*

Coordina lanzamientos regulares y programados (como semanales o mensuales) en los que las funciones se agrupan para su implementación.

Exploremos Git Flow porque su estructura clara lo hace excelente para comprender cómo los equipos colaboran con el control de versiones. Esta estrategia de ramificación organiza el trabajo utilizando diferentes tipos de ramas, cada una con un propósito específico. Algunas de

las ramas permanecerán durante todo el ciclo de vida del proyecto, mientras que otras solo existirán el tiempo suficiente para completar una tarea concreta.

Exploraremos cada tipo de rama y aprenderemos cómo los equipos trabajan juntos sin pisarse unos a otros.

## Ramas principales

Git Flow se basa en dos ramas principales de larga duración que constituyen la base de tu proyecto.

La rama *principal* representa tu código listo para producción. Es tu fuente de verdad. Esta rama siempre debe ser estable y lista para la implementación. Cuando trabajas en equipo:

*Mantén la rama principal protegida de las confirmaciones directas*

> En Git Flow, la rama principal recibe código que ya ha sido revisado y probado a través de la rama de desarrollo, por lo que la protección aquí se centra en evitar confirmaciones directas accidentales en lugar de en la revisión del código.

*Asegúrate de que la rama principal supere todas las pruebas automatizadas*

> Las pruebas automatizadas son tu red de seguridad para detectar problemas antes de que lleguen a la producción.

**NOTA**

La rama principal se denominaba tradicionalmente *«master»*, pero muchas organizaciones utilizan ahora *«main»* para adoptar una terminología más inclusiva. Ambos términos se refieren al mismo concepto.

---

La rama *de desarrollo* es donde se desarrolla la actividad diaria en Git Flow. Piensa en ella como el espacio de trabajo compartido de tu equipo, donde se reúnen las características individuales antes de que estén listas para la producción.

Cuando trabajes con la rama de desarrollo:

*Protege la rama develop con requisitos de revisión de código*

: Aquí es donde realmente se lleva a cabo la revisión de código en Git Flow. Cuando las ramas de características se fusionan en develop, los miembros del equipo revisan los cambios, detectan problemas y garantizan los estándares de codificación.

*Mantén la rama develop siempre compilable*

: Aunque pueda contener características incompletas, el código siempre debe compilarse y superar las pruebas básicas.

*Prueba la integración con regularidad*

: Dado que «develop» contiene múltiples características en las que se trabaja simultáneamente, ejecuta con frecuencia tu conjunto completo de pruebas para detectar problemas de integración de forma temprana.

La rama de desarrollo actúa como un área de preparación donde se prueban las características juntas antes de empaquetarlas en una versión y, finalmente, fusionarlas con la principal.

### Ramas de apoyo

Además de las ramas principales, Git Flow utiliza ramas temporales y de corta duración para tareas específicas. Estas ramas de apoyo ayudan a mantener el trabajo organizado y evitan conflictos entre los miembros del equipo.

Las ramas de características son donde pasarás la mayor parte de tu tiempo de desarrollo. Cuando empiezas a trabajar en una nueva característica o corrección de errores, creas una rama de características dedicada a partir de la rama de desarrollo. Esto aísla tu trabajo para que puedas experimentar y realizar cambios sin afectar a otros desarrolladores ni al código estable.

Dar a estas ramas nombres claros indica a otros desarrolladores (y a ti mismo en el futuro) qué trabajo se está realizando en esa rama. La tabla 10-1 muestra ejemplos de nombres de ramas buenos y malos.

Tabla 10-1. Ejemplos de nombresde ramas

| Buenos nombres de ramas | Nombres de ramas inadecuados |
| --- | --- |
| • feature/user-authentication<br>• corrección de errores/tiempo de espera de inicio de sesión<br>• mejora/optimización-del-rendimiento<br>• característica/JIRA-123-autenticación-de-usuario (cuando se utiliza el seguimiento de tickets)<br>• corrección de errores/GH-456-tiempo de espera de inicio de sesión (número de incidencia de GitHub) | • mi-rama<br>• cosas<br>• característica/123 (sin contexto sobre a qué se refiere 123) |

Más allá de las ramas de características, es posible que te preguntes: «¿Por qué no fusionar directamente el desarrollo con el principal cuando estés listo para lanzar?». Esa es una muy buena pregunta. Las ramas de lanzamiento resuelven varios problemas que pueden surgir cuando se trabaja en un entorno de equipo:

*Período de estabilización*

Mientras la rama de lanzamiento se somete a las pruebas finales y a la corrección de errores, tu equipo puede seguir añadiendo nuevas características al desarrollo para la próxima versión.

*Preparación del lanzamiento*

Las actualizaciones del número de versión, la documentación final y los cambios en la configuración de la implementación necesitan un espacio dedicado que no interfiera con el desarrollo en curso.

*Controles de calidad*

Muchas organizaciones requieren una fase de «candidato a lanzamiento» en la que las partes interesadas puedan aprobar el código exacto que se utilizará en la producción.

*Lanzamientos programados*

Si tu equipo lanza versiones cada dos semanas, necesitas una forma de «congelar» un conjunto de funciones mientras continúas con el desarrollo para el siguiente ciclo.

No todos los equipos necesitan ramas de lanzamiento; realmente depende de las prácticas de tu organización. Cuando tu equipo tiene un proceso de lanzamiento formal o ventanas de implementación programadas, las ramas de lanzamiento proporcionan una buena separación.

También puedes tener ramas de hotfixes e . Quizás también te preguntes: «¿Por qué necesito una rama separada para un hotfix cuando puedo usar develop?». ¿Qué sucede cuando la producción tiene un problema grave, pero tu rama develop contiene características a medio terminar que no están listas para la producción?

Las ramas de hotfix resuelven esta situación de emergencia haciendo lo siguiente:

*Omisión de código inestable*

Puedes solucionar el problema de producción sin implementar funciones incompletas desde el desarrollo.

*Mantener un historial limpio*

La corrección se aplica tanto a la rama principal (para su implementación inmediata) como a la rama de desarrollo (para que no se pierda en futuras versiones).

*Velocidad y enfoque*

Tu equipo puede trabajar en la corrección crítica sin preocuparse por otros trabajos de desarrollo en curso.

*Riesgo mínimo*

Dado que las revisiones se derivan de la rama principal estable, solo se cambia lo que es absolutamente necesario.

En esta sección, has aprendido sobre los tipos de ramas en Git Flow. Como puedes ver, cada tipo de rama tiene un propósito específico para mantener el trabajo de tu equipo organizado y correctamente versionado. Las ramas principales (main y develop) proporcionan estabilidad y coordinación, mientras que las ramas de apoyo (feature, release y hotfix) se encargan de tareas específicas sin interrumpir el flujo de trabajo principal. Al principio puede parecer abrumador, pero cuanto más trabajes con una estrategia como esta, más verás cómo crea límites claros que ayudan a los equipos a trabajar juntos de forma segura.

## El ciclo de vida del control de versiones

Entonces, ¿cómo funcionan todas estas ramas juntas para formar una estrategia de control de versiones? En Git Flow y en muchas otras es-

trategias, el desarrollo se produce en ramas de características, se fusiona en una rama de desarrollo y, finalmente, se coloca en la rama principal después del lanzamiento, con hotfixes aplicados directamente tanto a la rama maestra como a la de desarrollo cuando es necesario.

El flujo de trabajo de Git Flow se ilustra en la figura 10-1.

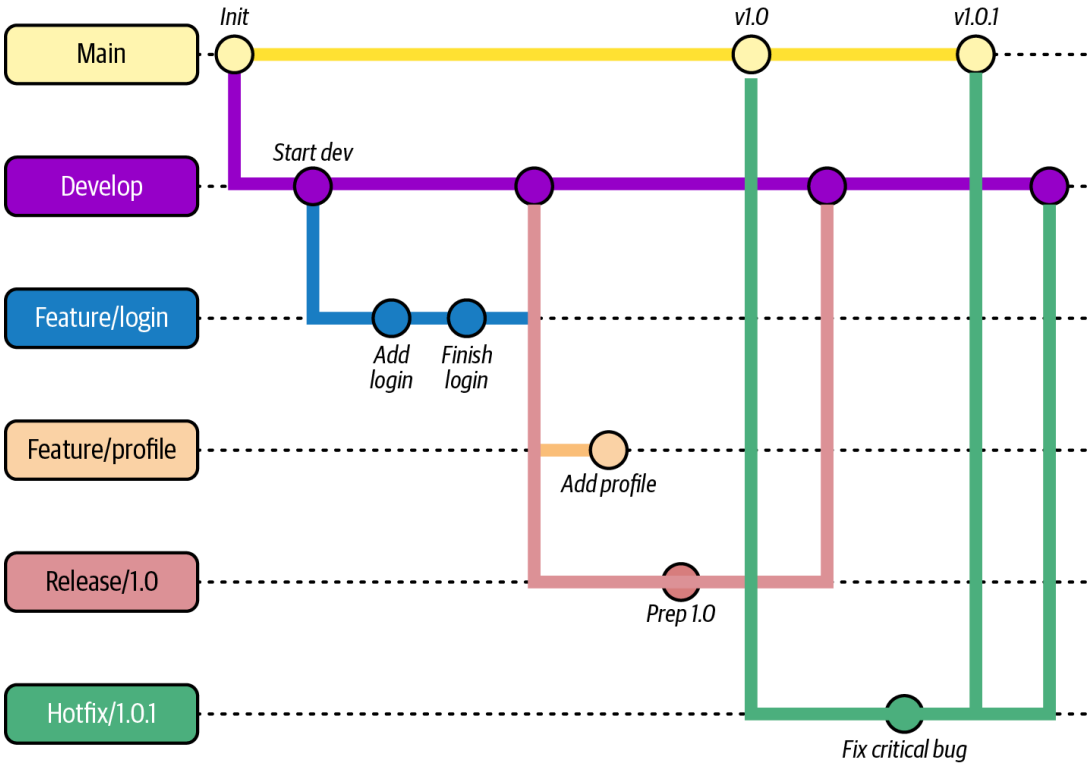



Figura 10-1. Flujo de trabajo deGit Flow

Al comprender Git Flow, tendrás una base sólida para el desarrollo en equipo, pero recuerda que es solo uno de los muchos enfoques posibles. A medida que trabajes con diferentes equipos y proyectos, encontrarás variaciones de estas estrategias, pero los objetivos seguirán siendo los mismos. Proporcionar una forma de proteger tu código, aislar tu trabajo y crear un proceso claro para integrar los cambios. En la si-

guiente sección, exploraremos cómo automatizar las implementaciones para garantizar que tu código, cuidadosamente gestionado, llegue a la producción de forma fiable y coherente.

## Automatización de la implementación

Las implementaciones manuales e es a producción suelen requerir mucho tiempo, son propensas a errores y resultan estresantes. Esto se debe a que es necesario reproducir una implementación paso a paso con intervención humana, lo que implica la preparación y el empaquetado del código, las pruebas y la verificación, la preparación del servidor, la ejecución de la implementación, la verificación posterior a la implementación y mucho más. La buena noticia es que podemos abordar este problema automatizando nuestras implementaciones. En esta sección, aprenderás varios elementos clave que te ayudarán a optimizar tu proceso de implementación.

### Implementaciones mediante scripts

Las implementaciones manuales son como fabricar a mano una estantería personalizada cada vez que necesitas una. Incluso si has construido la misma estantería que antes, es probable que te saltes un paso, hagas pequeñas variaciones o tardes mucho más de lo necesario. La automatización de la implementación es como tener una fábrica que puede construir estanterías idénticas según tus especificaciones exactas: precisas, repetibles y sin errores humanos. Un script de implementación sirve como instrucciones de fábrica, documentando cada paso

necesario para trasladar de forma coherente tu aplicación del desarrollo a la producción.

A continuación se muestra un ejemplo sencillo de un script de implementación que ejecutará pasos como crear directorios, realizar copias de seguridad de los directorios existentes, implementar la aplicación e iniciarla:

```
#!/bin/bash

# Configuration
APP_NAME="myapp"
JAR_FILE="target/${APP_NAME}.jar"
DEPLOY_DIR="/opt/applications/${APP_NAME}"
BACKUP_DIR="${DEPLOY_DIR}/backups"
LOG_FILE="${DEPLOY_DIR}/app.log"
PID_FILE="${DEPLOY_DIR}/${APP_NAME}.pid"
JVM_OPTS="-Xmx512m -Xms256m"

# Create directories
mkdir -p "${DEPLOY_DIR}" "${BACKUP_DIR}"

# Backup existing deployment
if [ -f "${DEPLOY_DIR}/${APP_NAME}.jar" ]; then
    mv "${DEPLOY_DIR}/${APP_NAME}.jar" \
    "${BACKUP_DIR}/${APP_NAME}-$(date +'%Y%m%d_%H%M%S').jar"
fi

# Stop running application
if [ -f "${PID_FILE}" ]; then
    kill $(cat "${PID_FILE}") 2>/dev/null || true
    rm -f "${PID_FILE}"
```

```
    sleep 2
fi

# Deploy new version
if [ -f "${JAR_FILE}" ]; then
    cp "${JAR_FILE}" "${DEPLOY_DIR}/${APP_NAME}.jar"
else
    echo "Error: JAR file not found: ${JAR_FILE}"
    exit 1
fi

# Start application
nohup java ${JVM_OPTS} -jar "${DEPLOY_DIR}/${APP_NAME}.jar" \
    > "${LOG_FILE}" 2>&1 &
echo $! > "${PID_FILE}"

# Quick health check
sleep 5
if kill -0 $(cat "${PID_FILE}") 2>/dev/null; then
    echo "Deployment successful. PID: $(cat ${PID_FILE})"
else
    echo "Deployment failed. Check ${LOG_FILE} for details."
    exit 1
fi
```

Aunque este ejemplo es básico, ilustra los componentes clave de la automatización de la implementación:

- Pasos claros y documentados
- Gestión de errores
- Procedimientos de copia de seguridad

- Enfoque sistemático

Incluso un script sencillo es mejor que no tener ningún script. Empieza poco a poco y mejora gradualmente la automatización de la implementación con el tiempo. Tu yo futuro te agradecerá que dediques tiempo a automatizar este proceso ahora.

### Procedimientos de reversión

Hemos establecido que la automatización de la implementación ahorrará tiempo y reducirá el estrés. Sin embargo, incluso con una implementación automatizada bien probada, en ocasiones pueden surgir problemas.

Nada es perfecto, por lo que debes estar preparado para problemas inesperados. Tener un plan para revertir rápidamente los cambios a un estado funcional garantiza que tus clientes puedan seguir utilizando tu aplicación sin interrupciones.

A continuación se muestra un ejemplo de un script de reversión que buscará la copia de seguridad más reciente, detendrá la aplicación, la restaurará, la iniciará y, a continuación, verificará que la aplicación está en funcionamiento y funciona correctamente:

```
#!/bin/bash

# Configuration (should match deploy.sh)
APP_NAME="myapp"
DEPLOY_DIR="/opt/applications/${APP_NAME}"
```

```
BACKUP_DIR="${DEPLOY_DIR}/backups"
LOG_FILE="${DEPLOY_DIR}/app.log"
PID_FILE="${DEPLOY_DIR}/${APP_NAME}.pid"
JVM_OPTS="-Xmx512m -Xms256m"

# Find most recent backup
LATEST_BACKUP=$(ls -t ${BACKUP_DIR}/${APP_NAME}-*.jar 2>/dev/null | head -1)

if [ -z "${LATEST_BACKUP}" ]; then
    echo "Error: No backup found in ${BACKUP_DIR}"
    exit 1
fi

# Stop current application
if [ -f "${PID_FILE}" ]; then
    kill $(cat "${PID_FILE}") 2>/dev/null || true
    rm -f "${PID_FILE}"
    sleep 2
fi

# Restore backup
cp "${LATEST_BACKUP}" "${DEPLOY_DIR}/${APP_NAME}.jar"

# Start application
nohup java ${JVM_OPTS} \
    -jar "${DEPLOY_DIR}/${APP_NAME}.jar" \
    > "${LOG_FILE}" 2>&1 &
echo $! >  "${PID_FILE}"

# Quick health check
sleep 5
if kill -0 $(cat "${PID_FILE}") 2>/dev/null; then
    echo "Rollback successful. Restored: $(basename ${LATEST_BACKUP})"
```

```
        echo "Application running with PID: $(cat ${PID_FILE})"
    else
        echo "Rollback failed. Check ${LOG_FILE} for details."
        exit 1
    fi
```

Antes de implementar un script de reversión, debes cumplir estos requisitos previos:

- Verifica que existan versiones de copia de seguridad antes de la implementación.
- Prueba los procedimientos de reversión con regularidad.
- Documenta los pasos de reversión de la migración de la base de datos.
- Mantén varias versiones de copia de seguridad.
- Monitorea el sistema durante la reversión.

El objetivo de la automatización de la implementación no es solo ahorrar tiempo. Se trata de crear un proceso fiable y repetible que te dé confianza en tus implementaciones. Empieza poco a poco, tal vez con un script sencillo que automatice unos pocos pasos, y ve avanzando gradualmente hacia una automatización más completa. Cada mejora reduce el riesgo y te acerca al ideal de implementaciones de e es predecibles y sin estrés.

## Estrategias de implementación

, no existe una estrategia única para implementar tu código en producción. La estrategia que elijas para tu aplicación puede marcar la dife-

rencia entre una implementación fluida y estar de guardia todo el fin de semana. Veamos algunos de los enfoques más populares y cuándo elegirlos.

## Implementación total (big bang)

Esta estrategia de implementación de gran alcance ( ) sustituye toda la aplicación de una sola vez. La versión antigua se sustituye por completo por la nueva de inmediato. Es fácil de implementar, pero conlleva un alto riesgo, ya que cualquier problema afectará inmediatamente a todos los usuarios. No hay un período de transición gradual, lo que complica la reversión si surgen problemas. Este enfoque es más adecuado para aplicaciones más pequeñas que se han sometido a pruebas exhaustivas o en entornos en los que el tiempo de inactividad es aceptable.

La implementación big bang es la más adecuada para lo siguiente:

- Aplicaciones pequeñas con usuarios limitados
- Entornos de desarrollo y pruebas
- Sistemas que pueden tolerar cierto tiempo de inactividad

## Implementación gradual (enfoque por fases)

Una implementación gradual es como si tu restaurante favorito introdujera una nueva receta a un pequeño número de clientes cada vez. Esto permite al chef recabar opiniones, realizar iteraciones y ampliar gradualmente la disponibilidad de la receta. La implementación gradual sigue este enfoque al implementar los cambios en un subconjunto

de usuarios o servidores antes de su lanzamiento general. Este enfoque controlado permite detectar problemas de forma temprana y corregir el rumbo.

*Algunos* ejemplos comunes son*las implementaciones canarias (* que se implementan primero en un pequeño porcentaje de usuarios) y *las implementaciones progresivas* (que actualizan gradualmente los servidores uno por uno). Ambas pueden realizarse a menudo sin tiempo de inactividad utilizando técnicas como la infraestructura azul-verde o el desplazamiento del tráfico del equilibrador de carga.

La implementación gradual es la mejor opción para lo siguiente:

- Aplicaciones a gran escala
- Funciones que se benefician de los comentarios de los usuarios
- Sistemas que requieren un monitoreo cuidadoso

### Consideraciones sobre el tiempo de inactividad cero

Tanto si eliges una implementación total como una gradual, puedes minimizar o eliminar el tiempo de inactividad utilizando técnicas como la implementación azul-verde (mantener dos entornos idénticos y cambiar entre ellos) o las actualizaciones continuas. Aunque su configuración es más compleja, estas técnicas proporcionan la experiencia de usuario más fluida para aplicaciones críticas y servicios de alto tráfico.

### Elegir tu estrategia

¿Cuándo debes elegir cada enfoque? A estas alturas, probablemente ya hayas adivinado que la respuesta suele ser «depende». Ten en cuenta factores como la complejidad del sistema, la tolerancia al tiempo de inactividad y las capacidades de monitoreo a la hora de seleccionar la estrategia adecuada para tu aplicación.

Recuerda: no hay que avergonzarse de empezar por lo sencillo y evolucionar la estrategia de implementación a medida que la aplicación y el equipo maduran.

## Integración continua e implementación continua

En el capítulo 5, aprendiste que las pruebas automatizadas proporcionan confianza durante la escritura y la confirmación del código, al tiempo que ayudan a identificar problemas en las primeras fases del ciclo de vida del desarrollo. Pero, ¿cómo puedes asegurarte de que estas pruebas se ejecutan cada vez que se promueve una versión a un nuevo entorno, como el de producción? ¿Cómo garantizas que tu código se compila, prueba e implementa sin los errores que puede introducir la intervención humana manual?

Aquí es donde entra en juego *la integración continua y la implementación continua*, o CI/CD (o entrega). Se trata de un conjunto de procesos automatizados que llevan tu código desde el desarrollo hasta la producción, donde se encuentran tus usuarios. *La integración continua*

combina y prueba automáticamente los cambios de todo tu equipo de desarrollo. *La entrega continua* prepara tu código probado para su lanzamiento, mientras que *la implementación continua* puede lanzarlo automáticamente a producción.

Por ejemplo, imagina que es tu segunda semana en el trabajo y el equipo te ha pedido que ayudes con la implementación de un microservicio más pequeño. Abres el extenso documento de implementación que está lleno de pasos vagos como «crear la aplicación», «ejecutar las pruebas», «actualizar la base de datos» e «implementar en el servidor». Tu pánico aumenta a medida que vas marcando cada elemento, sabiendo que si te saltas incluso un pequeño paso, podrías paralizar la producción.

Suena bastante estresante, ¿verdad? Por eso precisamente necesitas la automatización. La automatización puede eliminar muchos problemas al crear un proceso repetible y fiable. Las ventajas de la automatización incluyen lo siguiente:

*Coherencia*

¿Recuerdas la situación de «funciona en mi máquina», en la que la aplicación funciona localmente pero tiene problemas cuando se traslada a otro entorno? Cuando automatizas el proceso de compilación, minimizas ese problema.

*Pruebas automatizadas*

En lugar de depender de los desarrolladores para ejecutar un conjunto de pruebas antes de confirmar su código, tu proceso de CI/CD puede ejecutar pruebas automáticamente.

*Menos estrés*

En lugar de preocuparte por comprobar manualmente cada paso del proceso de implementación, puedes estar seguro de que tu automatización se ejecuta de la misma manera cada vez. Un script nunca se salta un paso ni escribe mal un comando.

*Detección temprana de problemas de integración*

Un proceso de CI/CD bien definido puede evitar que los problemas se cuelen en la producción, lo que te ahorrará tiempo y dinero.

*Reducción del tiempo entre el código y la producción*

Pasa a producción con frecuencia aprovechando un proceso automatizado bien definido que minimiza el tiempo de implementación. Recuerda que el código no tiene valor real hasta que llega a los usuarios finales.

## Creación de un flujo de trabajo básico de CI/CD

Un flujo de trabajo de CI/CD es como una receta: define los pasos necesarios para llevar tu código desde el desarrollo hasta la producción. Veamos un flujo de trabajo básico:

*Código*

Un desarrollador envía el código al repositorio.

*Compilación*

La aplicación se compila y se construye.

*Prueba*

Se ejecutan pruebas automatizadas.

*Paquete*

La aplicación se empaqueta para su implementación.

*Implementar*

La aplicación se implementa en el entorno de destino.

Para ponerlo en práctica, creemos un flujo de trabajo básico de GitHub Actions . Un *flujo de trabajo de GitHub* es un proceso automatizado que puedes configurar en tu repositorio de GitHub para compilar, probar, empaquetar, publicar o implementar tu código. GitHub Actions es una opción muy popular para implementar canalizaciones de CI/CD porque es fácil de usar e integra de forma natural con tu repositorio de GitHub.

A continuación se muestra un archivo de flujo de trabajo de ejemplo que implementa estos pasos para una aplicación Java:

```
name: Java CI/CD Pipeline

on:
 push:
   branches: [ main ]
 pull_request:
   branches: [ main ]

jobs:
 build-and-test:
```

```
runs-on: ubuntu-latest

steps:
# Check out your repository code
- uses: actions/checkout@v4

# Set up Java development environment
- name: Set up JDK 25
  uses: actions/setup-java@v4
  with:
    java-version: '25'
    distribution: 'temurin'

# Build the application
- name: Build with Maven
  run: ./mvnw -B package --file pom.xml

# Run automated tests
- name: Run tests
  run: ./mvnw test

# Package application (creates a JAR file)
- name: Package application
  run: ./mvnw package -DskipTests

# Store the package as an artifact
- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: app-package
    path: target/*.jar
```

Este archivo de flujo de trabajo muestra varios conceptos clave:

- La sección « `on` » define cuándo se ejecuta el flujo de trabajo (al enviarlo al principal o a las solicitudes de incorporación de cambios).
- Cada trabajo se ejecuta en un entorno virtual nuevo ( `ubuntu-latest` ).
- Los pasos se ejecutan de forma secuencial, y cada paso depende del éxito de los pasos anteriores.
- El flujo de trabajo se encarga de compilar, probar y empaquetar nuestra aplicación.

Al implementar este flujo de trabajo, has automatizado varios pasos críticos en tu proceso de desarrollo. Cuando envías código o creas una solicitud de incorporación de cambios, GitHub realiza automáticamente lo siguiente:

- Compila tu aplicación para detectar errores de compilación de forma temprana.
- Ejecuta tu conjunto de pruebas para detectar posibles errores.
- Crea un paquete listo para implementar.
- Almacena el paquete para su uso posterior.

Un flujo de trabajo de CI/CD automatiza el recorrido del código desde el desarrollo hasta la producción a través de una serie de pasos predefinidos. Esta automatización elimina la intervención manual, detecta los errores de forma temprana, aplica los estándares de calidad y, en última instancia, entrega el software de forma más fiable y eficaz siguiendo un patrón repetible.

### Patrones avanzados de CI/CD

No te estreses por tener que aprenderlo todo de golpe. Empieza por lo básico y construye sobre los fundamentos que has aprendido en este capítulo. Comienza automatizando tu proceso de compilación y prueba. A medida que empiece a tener más sentido, te sentirás más seguro y podrás añadir gradualmente estrategias de implementación más avanzadas.

A medida que avances en tu carrera, te encontrarás con patrones de CI/CD más sofisticados como estos:

*Lanzamientos canarios*

: Implementa primero el nuevo código en un pequeño subconjunto de usuarios o servidores para probarlo en producción con un riesgo mínimo y amplíalo gradualmente si no se detectan problemas.

*Implementaciones azul-verde*

: Mantén dos entornos de producción idénticos (azul y verde), con solo uno activo a la vez. Las nuevas versiones se implementan en el entorno inactivo y, una vez verificadas, el tráfico se cambia por completo.

A continuación, te mostramos algunas formas de introducir patrones avanzados y mejorar gradualmente tu proceso de CI/CD:

1. Comienza con un proceso básico que compila y prueba tu código.
2. Incorporar gradualmente más comprobaciones y validaciones automatizadas.

3. Mantén un proceso rápido para garantizar ciclos de retroalimentación rápidos.
4. Realiza un monitoreo regular y optimiza tu canalización en función de las necesidades del equipo.
5. Documenta tu canalización para facilitar la colaboración del equipo.

CI/CD no se trata solo de herramientas y automatización. Se trata de fomentar una cultura de mejora continua y entrega de software confiable. Al adoptar estas prácticas al inicio de tu carrera, construirás una base sólida para tu crecimiento profesional y te convertirás en un miembro invaluable del equipo.

## Monitoreo y mantenimiento del sistema de producción

Has escrito tu código, lo has probado a fondo y ahora está listo para la producción. Tu aplicación ha pasado por el canal CI/CD adecuado y ahora está en producción. Es hora de descansar y relajarse, ¿verdad? No del todo. El camino de una aplicación que funciona en tu máquina hasta convertirse en un sistema fiable en producción es continuo.

### Monitoreo

Un monitoreo y un registro adecuados de la e idad pueden decirte mucho sobre el estado de tu aplicación. Sin ellos, no sabrás cómo está funcionando tu sistema actualmente o a lo largo del tiempo. Demasiados

desarrolladores quieren lanzar su aplicación a producción y esperar lo mejor. Piensan que no tener noticias es una buena noticia. La verdad es que los usuarios rara vez te dicen cuando algo va mal. Simplemente se van.

Tu aplicación se basa en dos tipos de sistemas de monitoreo:

*Monitoreo en tiempo real*

Realiza un seguimiento de lo que está sucediendo en este momento. ¿El tiempo de respuesta de tu aplicación es demasiado lento? ¿La carga o el rendimiento del servidor son demasiado altos? Las métricas individuales nunca cuentan toda la historia. Un tiempo de respuesta de 300 ms puede parecer rápido, pero si tu aplicación normalmente responde en 50 ms, esto podría indicar un problema.

*Registros*

Sirven como registro histórico. Cuando surgen problemas, unos buenos registros te ayudan a reconstruir qué falló y cuándo. Proporcionan el contexto para la resolución de problemas.

Después de experimentar varios incidentes críticos con nuestra aplicación, establecimos estas reglas de monitoreo fundamentales:

1. Registra la información que necesitarás cuando surjan problemas.
2. Incluye siempre marcas de tiempo e identificadores de usuario.
3. Marca los errores como errores. No ocultes alertas importantes en los registros de información.
4. Mantén los datos privados fuera de los registros (sin contraseñas ni datos personales).

5. Elimina los registros antiguos antes de que consuman tu almacenamiento.

---

**CONSEJO**

Al principio, mantén tu sistema de monitoreo sencillo. Vigila los tiempos de respuesta, el número de errores y los recursos del servidor. Eso es suficiente para empezar. Las herramientas sofisticadas pueden venir después.

---

Un buen monitoreo proporciona tranquilidad. Los problemas acabarán surgiendo; esa es la naturaleza del software. Cuando lo hagan, agradecerás disponer de métricas y registros que te ayuden a diagnosticar y solucionar los problemas rápidamente.

---

**NOTA SOBRE IA**

Una vez que tengas los registros y las métricas e , las herramientas de IA pueden mejorar significativamente tus capacidades de monitoreo. La IA puede buscar en grandes volúmenes de registros para identificar anomalías o patrones que puedan indicar problemas emergentes antes de que se conviertan en críticos. Además, la IA puede ayudar a generar paneles de control de monitoreo a partir de tus datos métricos, lo que tradicionalmente requiere una tediosa configuración manual, lo que te proporciona un punto de partida sólido y acelera el proceso de creación de paneles.

---

# Mantenimiento del sistema

Cuando se implementa un sistema en producción, no es el final del viaje, sino solo el comienzo. El mantenimiento del sistema es un aspecto fundamental para que el software funcione de manera eficiente, segura y confiable. En esta sección se explica cómo mantener el sistema de producción para evitar problemas costosos y garantizar un rendimiento óptimo.

## Mantener los sistemas actualizados

El software que se ha implementado en producción necesita un mantenimiento regular. Aunque todo funcione bien en producción, puede ser tentador posponer el mantenimiento, pero esto sería un error. Lo que puede empezar como un problema menor y poco costoso puede convertirse rápidamente en un problema grave y costoso.

Cuando se trata de sistemas de los que dependen los usuarios, los costes son elevados. Las infraestructuras críticas requieren un mantenimiento constante para garantizar su seguridad y fiabilidad. Sin embargo, en el desarrollo de software, los equipos suelen retrasar las actualizaciones necesarias a pesar de los riesgos.

## Actualizaciones del sistema

Las actualizaciones del sistema operativo y los parches de seguridad constituyen tu primera línea de defensa para evitar problemas graves y costosos. Estas actualizaciones suelen abordar lo siguiente:

- Vulnerabilidades críticas de seguridad
- Mejoras en el rendimiento
- Correcciones de errores
- Nuevas funciones y capacidades

Nunca ignores las actualizaciones de seguridad. Aunque las actualizaciones de funciones a veces pueden esperar, los parches de seguridad deben aplicarse de forma prompt para proteger tus sistemas de vulnerabilidades.

### Gestión de dependencias

La gestión de dependencias es posiblemente el aspecto más descuidado del mantenimiento del sistema. Tu aplicación puede depender de docenas o incluso cientos de bibliotecas externas, cada una de las cuales puede albergar vulnerabilidades de seguridad. ¿Recuerdas las SBOM que hemos comentado anteriormente en este capítulo? Esta es precisamente la razón por la que es importante mantener una SBOM precisa para realizar un seguimiento y actualizar tus dependencias.

El siguiente código muestra una dependencia en una aplicación Java que declara una versión obsoleta con vulnerabilidades conocidas:

```
// Example dependency in pom.xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>2.5.5</version>  // Outdated version with known vulnerabilities
</dependency>
```

Cuando generas una SBOM para esta aplicación, las herramientas de seguridad pueden escanearla con bases de datos de vulnerabilidades para identificar que la versión 2.5.5 tiene problemas de seguridad conocidos y recomendar la actualización a una versión más reciente y segura.

Las herramientas de compilación modernas, como Maven y Gradle, ofrecen analizadores de dependencias que pueden alertarte de vulnerabilidades conocidas. Para detectar estos problemas a tiempo, configura análisis automatizados. En este ejemplo, una acción de GitHub ejecuta comprobaciones de dependencias en tu código:

```
// Example GitHub Actions workflow snippet
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run Dependency Check
        uses: dependency-check/Dependency-Check@main
```

### Tomar medidas

Puedes empezar a mejorar el mantenimiento del sistema estableciendo una política de actualización para tu equipo o proyecto:

1. Establece intervalos de actualización periódicos.
2. Define procedimientos de parches de emergencia.
3. Implementar el escaneo automatizado.

4. Documentar los procesos de actualización.
5. Mantener un registro de actualizaciones.

Recuerda que la prevención cuesta menos que la recuperación. Haz que las actualizaciones del sistema sean una prioridad en tu flujo de trabajo de desarrollo. Configura recordatorios en el calendario para revisar las actualizaciones periódicamente. A pesar de la automatización, la supervisión humana sigue siendo crucial para mantener el buen funcionamiento del sistema.

## Conclusión

Trasladar tu código de la seguridad de tu máquina local al impredecible mundo de la producción puede resultar intimidante al principio. Solo tienes que saber que no estás solo y que la mayoría de los desarrolladores han sentido ese mismo estrés en algún momento de su carrera. La preparación es la clave para garantizar un buen proceso de implementación. Llevar el código a producción es una habilidad que mejorará con el tiempo y la experiencia.

Recuerda estas conclusiones clave:

- Piensa en la producción desde el primer día. Ten en cuenta el rendimiento, la seguridad y el manejo de errores mientras programas.
- Configura tus aplicaciones correctamente para diferentes entornos utilizando variables de entorno e indicadores de características.
- Monitorea el estado de tu aplicación y configura un registro adecuado, pero nunca registres datos confidenciales.

- Utiliza estrategias de control de versiones como Git Flow para colaborar de forma segura con tu equipo.
- Automatiza tus implementaciones con CI/CD para que sean fiables y repetibles.
- Recuerda que a tus usuarios no les importa si la aplicación funciona en tu máquina. Les importa si funciona en la suya.

Cuando comiences a implementar tus propias aplicaciones en producción, te enfrentarás a retos y, a veces, incluso podrías fracasar: no pasa nada. Cada fracaso que encuentres es una oportunidad para aprender algo nuevo y te convertirá en un mejor ingeniero de software.

Concéntrate en crear buenos hábitos ahora, y las implementaciones en producción se convertirán en una parte natural de tu flujo de trabajo de desarrollo. En poco tiempo, serás un desarrollador experimentado que ayuda a otros a enviar código a la tierra prometida de la producción.

## Ponerlo en práctica

Es difícil imaginar cómo funcionará toda una aplicación en producción, así que adopta gradualmente algunas de las buenas prácticas que has aprendido en este capítulo. A continuación se indican algunos buenos hábitos que se pueden aplicar en varios momentos antes de la implementación en producción. La próxima vez que te asignen una función, elige uno o dos elementos de cada una de estas categorías para aplicarlos a tu trabajo. No intentes hacerlo todo a la vez, sino céntrate en crear estos hábitos gradualmente.

Antes de programar:

- Crea una lista sencilla de lo que podría fallar en tu función.
- Planifica qué información podrías necesitar registrar para depurar estos problemas.
- Identifica qué configuración podría cambiar de tu máquina local a otros entornos en tu flujo de trabajo.

Mientras programas:

- Crea una rama de características con un nombre claro y descriptivo antes de empezar a trabajar.
- Añade un manejo básico de errores para las tres cosas principales que podrían fallar.
- Incluye sentencias de registro para las acciones clave (¡pero evita registrar datos confidenciales!).
- Escribe una prueba que imite el comportamiento real de los usuarios.
- Utiliza valores de configuración para cualquier cosa que pueda cambiar entre entornos.

Antes de la implementación:

- Anota los pasos de implementación a medida que los realizas.
- Crea un plan de reversión sencillo. ¿Cómo desactivarías esta función si algo saliera mal?
- Pide a un compañero de equipo que revise tus cambios teniendo en cuenta la producción.

Después de la implementación:

- Comienza el monitoreo de tus registros después de la implementación durante una o dos horas. ¿Hay algo que llame la atención?
- Elabora una retrospectiva sobre esta implementación. Se trata de una reunión o debate que se celebra tras completar un proyecto o una implementación, en la que el equipo puede reflexionar sobre lo que ha salido bien, los retos a los que se ha enfrentado y las lecciones que se pueden extraer.

## Recursos adicionales

- *Continuous Delivery*, de Jez Humble y David Farley (Addison-Wesley Professional, 2010)
- *The Phoenix Project*, de Gene Kim et al. (IT Revolution Press, 2013)
- *Head First Git*, de Raju Gandhi (O'Reilly Media, 2022)
- *Learning GitHub Actions*, de Brent Laster (O'Reilly Media, 2023)
- *Feature Flags*, de Ben Nadel (autoeditado, Lulu, 2024)